WARD

Testes de unidade e integração simples, direto em Python

Disciplina: Teste de Software

Professor: Ítalo Linhares

Aluno: Luis Gustavo

A FERRAMENTA

O que é o Ward?

Ward é um framework de testes em Python, feito para testes de **unidade e integração**, pensado para ser mais simples e legível que os frameworks tradicionais.

Em vez de criar classes com métodos `test_algo(self)` como no modelo xUnit, no Ward cada teste é uma função Python comum marcada com um decorador e uma frase explicando o que o teste verifica.

LINK OFICIAL

`ward.readthedocs.io`

Repositório: github.com/darrenburns/ward

INSTALAÇÃO

Instalação da ferramenta

Requisito: Python 3.8+.

```
$ pip install ward
```

Depois de instalado, basta rodar o comando dentro da pasta do projeto que o Ward encontra os arquivos de teste sozinho:

```
$ ward test
```

SEM COMPLICAÇÃO

- ✓ Não precisa de classes nem heranca porque cada teste é uma função
- ✓ Não precisa de browser, driver ou servidor externo
- ✓ Descobre os testes sozinho, sem precisar configurar
- ✓ Funciona junto com qualquer código Python (funções, classes, módulos)

TIPO DE TESTE

Que tipo de teste ela executa?

Teste de Unidade

Verifica uma função ou classe de forma isolada
Por exemplo: um único método da classe Estoque.

Teste de Integração

Verifica se duas ou mais partes do sistema funcionam corretamente juntas
Por exemplo: Carrinho usando Estoque.

Teste Funcional

Verifica se os dois anteriores confirmam que a funcionalidade do sistema se comporta como esperado.

SISTEMA DE EXEMPLO

Sistema escolhido para os testes

Sistema de Estoque + Carrinho de Compras

classe Estoque (testada de forma isolada)

- `adicionar_produto(nome, qtd)`
- `remover_produto(nome, qtd)`
- `consultar_quantidade(nome)`

Controla quantos produtos existem. Nega quantidades inválidas e bloqueia remoções maiores que o disponível.

classe Carrinho (integração com Estoque)

- `adicionar_item(nome, qtd)`
- `total_itens()`
- `resumo()`

Quando adicionamos um item no carrinho, ele chama o Estoque para reservar (remover) a quantidade, por isso testar o Carrinho é um teste de INTEGRAÇÃO entre os dois módulos.

CRIANDO TESTES

Como criar um teste

Um teste no Ward é só uma função com o decorador @test e uma frase descrevendo o que ela verifica:

```
from ward import test, fixture, raises
from estoque import Estoque,ProdutoNaoEncontrado EstoqueInsuficiente
from carrinho import Carrinho

@fixture
def estoque_com_canetas():
    e = Estoque()
    e.adicionar_produto("Caneta", 10)
    return e


@test("remover mais do que o disponível gera EstoqueInsuficiente")
def _(estoque=estoque_com_canetas):
    with raises(EstoqueInsuficiente):
        estoque.remover_produto("Caneta", 999)
```

Para rodar os testes: `ward test`

RELATÓRIO

Como é o relatório de testes

Saída após rodar: ward test

```
Found 10 tests and 1 fixture

PASS  [UNIDADE] adicionar produto novo...     10%
PASS  [UNIDADE] remover quantidade válida...  30%
PASS  [UNIDADE-ERRO] estoque insuficiente...  50%
FAIL  [UNIDADE-BUG] remover qtd negativa...   70%
PASS  [INTEGRAÇÃO] carrinho reserva no est... 80%
PASS  [INTEGRAÇÃO-ERRO] compra bloqueada...  100%

AssertionError: Expected exception ValueError,
but None was raised instead.

=== 9 Passes (90%) · 1 Failure (10%) ===
```

O QUE O RELATÓRIO MOSTRA

- Uma linha por teste, com a frase descritiva e PASS/FAIL coloridos
- Uma barra de progresso com a porcentagem concluída
- Para falhas: o trecho de código exato e o valor que causou o erro
- Um resumo final com total de passes e falhas

EXECUÇÃO

Casos de sucesso

9 dos 10 testes passaram, pegando tanto o comportamento isolado do Estoque quanto sua integração com o Carrinho.

OK UNIDADE

`Estoque.adicionar_produto` `"Caderno", 20` **quantidade correta: 20**

OK UNIDADE

`Estoque.remover_produto` `"Caneta", 4 (de 10)` **quantidade correta: 6**

OK INTEGRAÇÃO

`Carrinho.adicionar_item` `"Caneta", 3` **carrinho=3 · estoque=7**

EXECUÇÃO

Casos de erro e os tratamentos

BUG ENCONTRADO COM O TESTE

`remover_produto("Caneta", -5)` → **estoque AUMENTA em vez de de ser rejeitado**

O teste escreveu o comportamento CORRETO esperado (rejeitar quantidade negativa) mas ele falhou: o código atual verifica se a quantidade é maior que o disponível, então um número negativo passa direto e a subtração vira soma.

TRATAMENTO APLICADO

Basta validar quantidade > 0 no início do remover_produto e lançar ValueError caso não dê certo, basta usar o mesmo padrão usado em adicionar_produto.

ERRO PROPAGADO CORRETAMENTE NA INTEGRAÇÃO

`Carrinho.adicionar_item("Caneta", 50)` com apenas 10 em estoque → `EstoqueInsuficiente`

Por que usar o Ward?

Simplicidade real

Sem classes, sem herança, sem boilerplate: cada teste é uma função com uma frase clara.

Unidade e integração

Serve tanto para testar uma função isolada quanto a interação entre módulos.

Descoberta automática

Basta rodar 'ward test' na pasta do projeto, não precisa fazer nenhuma configuração.

Achado de bugs reais

O teste de unidade encontrou uma falha real de validação no método remover_produto.